

Supplementary Material

Multilayer Graph Learning for Decreased Rank Attack Detection in RPL-Based IoT Networks

Muhammad Kazim and Sohail Abbas

Scope

This supplementary document contains technical details and expanded outputs that were compressed out of the eight-page IEEE Internet of Things Journal manuscript. The main paper reports the central controlled comparison. This document records the supporting Cooja trace protocol, mathematical details, complete result artifacts, ablation material, and reproduction commands.

Contents

1	Appendix I: Cooja Trace Generation and Parsing	2
1.1	Simulation Scope	2
1.2	DRA Mechanism	2
1.3	Parsed Tables and Leakage Control	2
2	Appendix II: Mathematical Model and Training Protocol	3
2.1	Multilayer Graph Representation	3
2.2	Layer Construction	3
2.3	Model Operations	3
2.4	Complexity and Equivariance	3
3	Appendix III: Complete Detection Outputs	5
4	Appendix IV: Layer Analysis, Ablation, and Propagation Sensitivity	8
4.1	Layer Structure	8
4.2	Layer Ablation	8
4.3	Propagation Sensitivity	9
5	Appendix V: Reproduction Commands	10
5.1	Environment	10
5.2	Parse Cooja Logs	10
5.3	Run Validation-Selected Benchmark	10
5.4	Run Layer Analysis and Propagation Sensitivity	10
5.5	Generate Tables and Figures	10

1 Appendix I: Cooja Trace Generation and Parsing

1.1 Simulation Scope

The benchmark uses Cooja/Contiki-NG traces rather than a purely synthetic feature generator. Each simulated network has one RPL root and 49 non-root motes. The decreased-rank attack (DRA) ratios are 10%, 20%, and 30%, corresponding to 5, 10, and 15 malicious motes, respectively. Five independent topology and attacker-placement seeds are used for each ratio, giving 15 Cooja logs. Each run lasts 600 s; the first 60 s are removed as warm-up, and the remaining trace is aggregated into 60 s observation windows. This produces 135 graph snapshots and 6750 node-level decisions.

1.2 DRA Mechanism

For node v_i , let R_i be its actual RPL rank and let R_i^{adv} be the rank advertised through DIO control messages. Under normal operation,

$$R_i^{adv} = R_i. \quad (1)$$

For a malicious mote $i \in \mathcal{M}$, the DRA implementation advertises an artificially lower rank:

$$R_i^{adv} = \max(R_{\min}, R_i - \delta_i), \quad \delta_i > 0. \quad (2)$$

A neighboring node v_j may then select a parent from its candidate set $\mathcal{N}_j$ using an objective that depends on advertised rank and link cost:

$$p(j) = \arg \min_{k \in \mathcal{N}_j} \Psi(R_k^{adv}, Q_{jk}). \quad (3)$$

This formulation makes DRA a routing-structure attack: the malicious mote does not need to jam the channel or modify packets directly; it changes parent-selection incentives by advertising a misleading rank.

1.3 Parsed Tables and Leakage Control

The parser separates the Cooja serial log into four artifact families:

- metadata for scenario ratio, seed, and attacker placement;
- periodic node traces, including rank, DAG rank, preferred parent, parent metric, RSSI, neighbor count, role, and scenario label;
- packet events, including transmission count, successful receptions, delivery delay, and packet-delivery ratio (PDR);
- DRA audit records, which verify when malicious motes advertise manipulated ranks.

The audit records are deliberately excluded from the model feature matrix and from all adjacency matrices. They are used only to verify that the attack is active and to audit the parser. Thus, the detector learns from observable node, routing, link, and packet behavior rather than from the internal attack hook.

2 Appendix II: Mathematical Model and Training Protocol

2.1 Multilayer Graph Representation

At each observation window, the network is represented as

$$\mathcal{G} = (V, \{A^{(\ell)}\}_{\ell \in \mathcal{L}}, X, Y), \quad \mathcal{L} = \{r, q, t, a\}, \quad (4)$$

where r , q , t , and a denote routing, link-quality, temporal, and trust/anomaly layers. The feature matrix $X \in \mathbb{R}^{N \times F}$ contains observable node and packet fields. For node v_i ,

$$x_i = [\tilde{i}, \tilde{R}_i, \tilde{G}_i, \tilde{S}_i, \tilde{M}_i, \tilde{N}_i, \tilde{D}_i, \tilde{T}_i, \tilde{P}_i], \quad (5)$$

where the terms are normalized node identifier, RPL rank, DAG rank, RSSI, parent metric, neighbor count, mean delay, transmission count, and PDR.

2.2 Layer Construction

The routing layer preserves DODAG parent-child support:

$$A_{ij}^{(r)} = \begin{cases} 1, & p(i) = j \text{ or } p(j) = i, \\ 0, & \text{otherwise.} \end{cases} \quad (6)$$

The link-quality layer uses the same support but weights edges by parent metric:

$$A_{ij}^{(q)} = \begin{cases} (\max\{M_i/M_0, 1\})^{-1}, & p(i) = j \text{ or } p(j) = i, \\ 0, & \text{otherwise.} \end{cases} \quad (7)$$

Temporal and trust/anomaly layers use similarity edges:

$$A_{ij}^{(\ell)} = \exp\left(-\frac{\|z_i^{(\ell)} - z_j^{(\ell)}\|_2^2}{\sigma_\ell^2}\right), \quad \ell \in \{t, a\}, \quad (8)$$

with only the strongest similarity links retained.

2.3 Model Operations

For each layer, self-loops are added and the adjacency is symmetrically normalized to obtain $\hat{A}^{(\ell)}$. A layer-specific GCN branch computes

$$H^{(\ell)} = \phi(\hat{A}^{(\ell)} X W^{(\ell)}). \quad (9)$$

ML-GCN fuses layer embeddings by fixed concatenation. Attn-ML-GCN learns a scalar attention weight per relation:

$$\alpha_\ell = \frac{\exp(\theta_\ell)}{\sum_{m \in \mathcal{L}} \exp(\theta_m)}, \quad H = [\alpha_r H^{(r)} \parallel \alpha_q H^{(q)} \parallel \alpha_t H^{(t)} \parallel \alpha_a H^{(a)}]. \quad (10)$$

Training minimizes weighted cross-entropy:

$$\mathcal{J}(\Theta) = - \sum_{i \in \mathcal{V}_{tr}} \sum_{c=0}^1 \omega_c \mathbb{I}(y_i = c) \log Z_{i,c}. \quad (11)$$

2.4 Complexity and Equivariance

For sparse matrices, the dominant forward cost of one ML-GCN block is

$$\mathcal{C}_{ML} = \mathcal{O}\left(\sum_{\ell \in \mathcal{L}} |E^{(\ell)}| F + |\mathcal{L}| N F d + N |\mathcal{L}| d\right), \quad (12)$$

where F is the input-feature dimension and d is the hidden dimension. Attention adds

$$\mathcal{C}_{attn} = \mathcal{O}(|\mathcal{L}|Nd + |\mathcal{L}|). \quad (13)$$

For any node permutation matrix P , the model satisfies

$$f_{\Theta}(PX, \{P\hat{A}^{(\ell)}P^T\}_{\ell \in \mathcal{L}}) = Pf_{\Theta}(X, \{\hat{A}^{(\ell)}\}_{\ell \in \mathcal{L}}), \quad (14)$$

so predictions depend on graph structure and features rather than arbitrary node ordering.

3 Appendix III: Complete Detection Outputs

Table 1 reports the held-out-seed result table used to derive the compressed table in the manuscript. The confusion-matrix figure is included to show decision counts for representative flat-feature, aggregate-graph, and multilayer graph models.

Table 1: Held-out-seed test performance.

Model	BAcc	F1	Recall	Precision	FPR
Logistic Regression	0.668	0.465	0.489	0.443	0.154
Random Forest	0.662	0.443	0.611	0.347	0.287
XGBoost	0.671	0.468	0.507	0.435	0.165
LightGBM	0.613	0.381	0.370	0.392	0.144
MLP	0.665	0.437	0.759	0.307	0.429
DQN	0.606	0.383	0.707	0.263	0.496
DDQN	0.527	0.343	0.937	0.210	0.882
Dueling DDQN	0.664	0.434	0.789	0.300	0.461
Agg-GCN	0.621	0.395	0.489	0.331	0.247
ML-GCN	0.721	0.531	0.626	0.460	0.183
Attn-ML-GCN	0.689	0.489	0.567	0.430	0.188

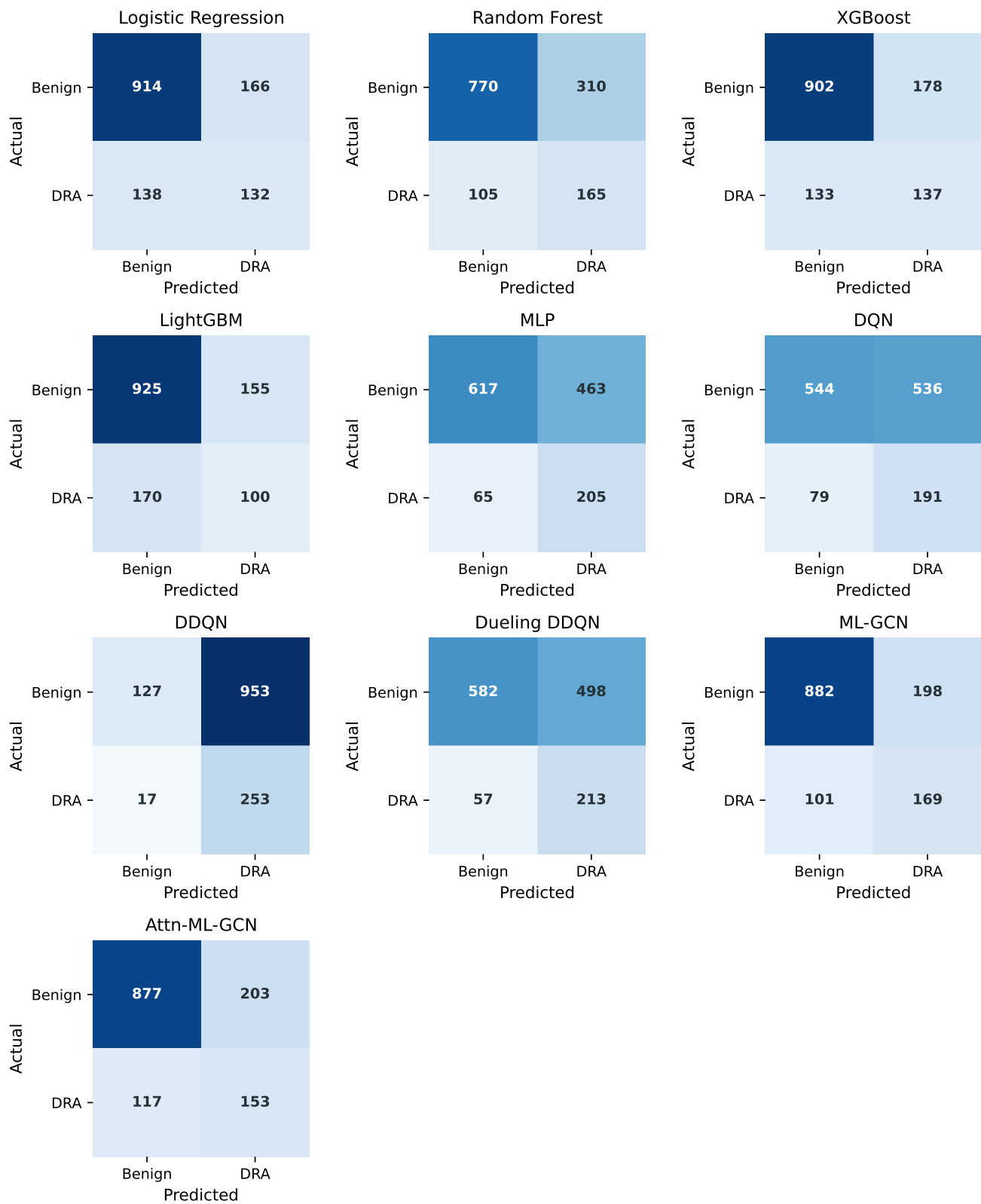


Figure 1: Complete confusion-matrix figure generated by the repository pipeline.

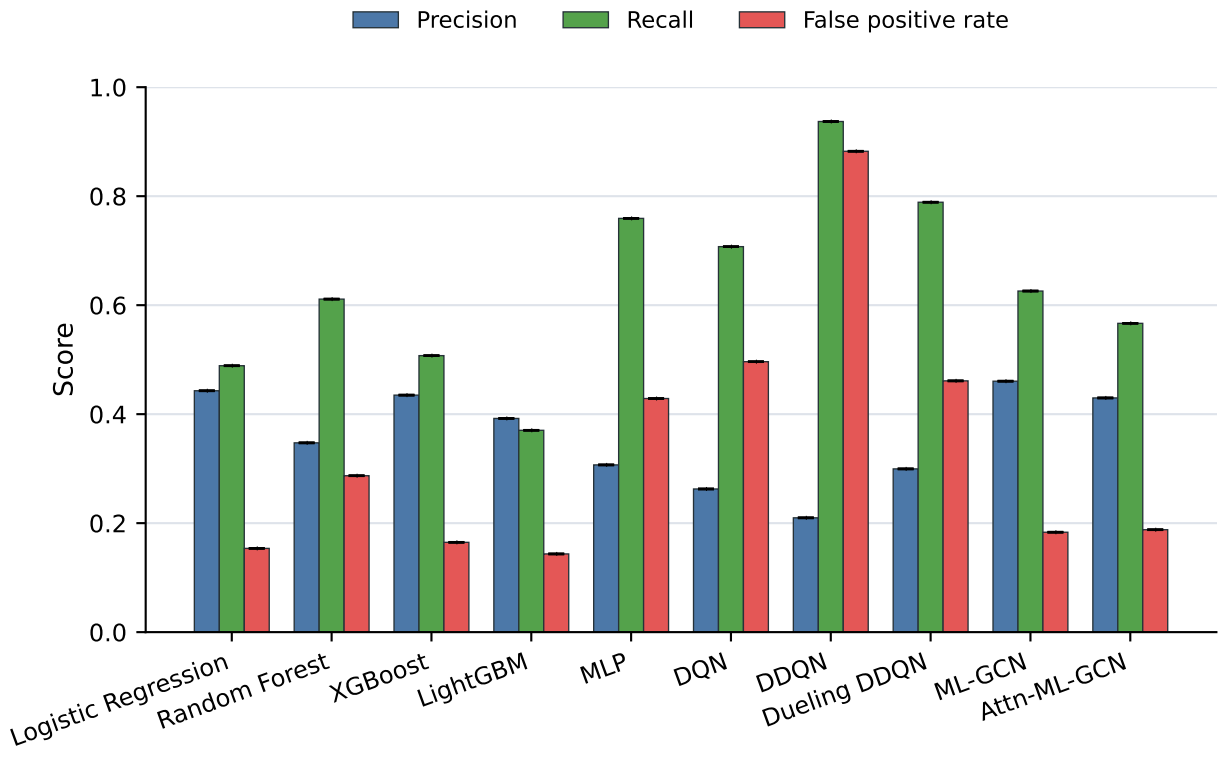


Figure 2: Precision, recall, and FPR comparison for the held-out-seed benchmark.

4 Appendix IV: Layer Analysis, Ablation, and Propagation Sensitivity

4.1 Layer Structure

The routing and link-quality layers are sparse parent-child supports. The temporal and trust/anomaly layers are denser because they encode behavioral similarity. Collapsing all layers into one support creates edge-support inflation and removes relation identity.

Table 2: Mean structural properties of the four RPL layers.

Layer	Mean edges	Density	Edge share
Routing	49.00	0.040	0.023
Link quality	49.00	0.040	0.023
Temporal	885.78	0.723	0.410
Trust/anomaly	1154.90	0.943	0.544

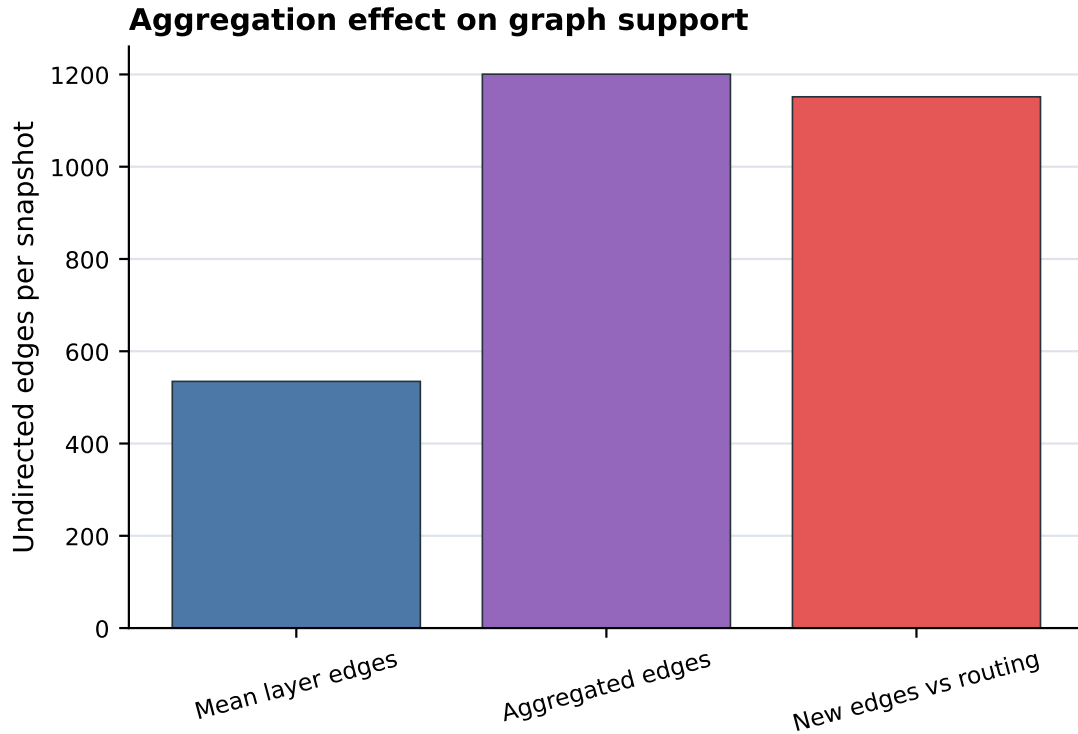


Figure 3: Aggregation-support analysis. Layer aggregation increases graph support and obscures relation identity.

4.2 Layer Ablation

Table 3 gives the expanded ablation values. The full multilayer model provides the best F1-score in the validation-selected manuscript setting, while routing-only and link-quality-only variants remain competitive because DRA directly targets parent selection.

Table 3: ML-GCN layer ablation results.

Variant	BAcc	F1	FPR
All layers (ML-GCN)	0.721	0.531	0.183
Routing only	0.719	0.509	0.265
Link-quality only	0.706	0.495	0.262
Without temporal	0.693	0.481	0.258
Without routing	0.677	0.469	0.212
Without link quality	0.671	0.471	0.150
Temporal only	0.525	0.337	0.801
Trust/anomaly only	0.517	0.341	0.967

4.3 Propagation Sensitivity

The graph sensitivity experiment evaluates one-hop and two-hop propagation. Two-hop propagation reduces performance for Agg-GCN, ML-GCN, and Attn-ML-GCN, suggesting that deeper propagation can overmix node-specific evidence in small LLN snapshots.

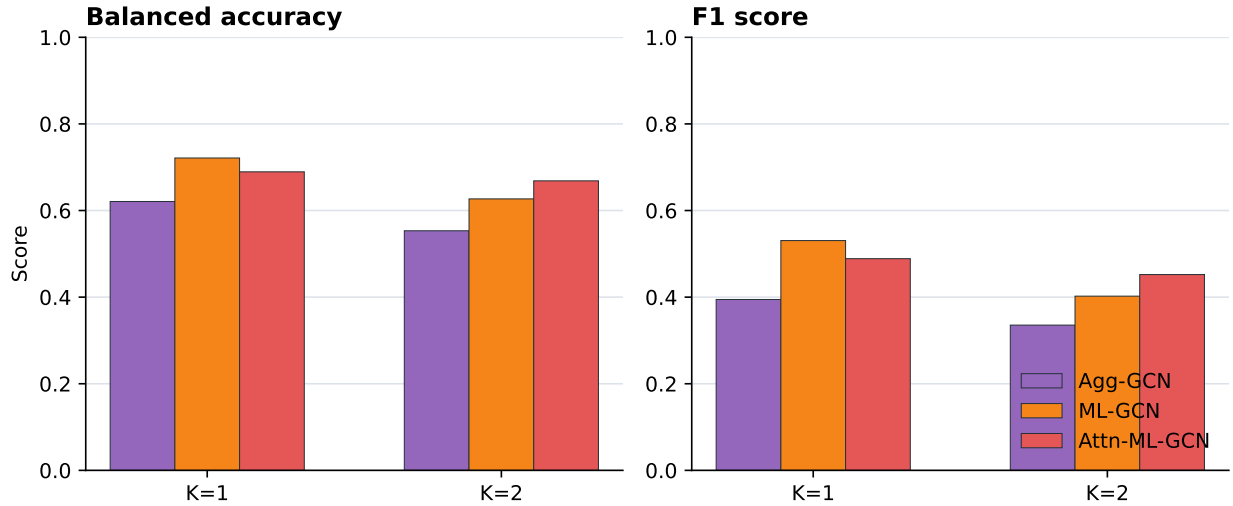


Figure 4: Propagation-depth sensitivity for graph-based models.

5 Appendix V: Reproduction Commands

5.1 Environment

```
python3 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
```

5.2 Parse Cooja Logs

```
python scripts/parse_cooja_logs.py \
--log-root "/path/to/cooja-contiki-ng/examples/rpl-dra-ml/generated" \
--output data/cooja
```

5.3 Run Validation-Selected Benchmark

```
python scripts/run_cooja_experiment.py \
--snapshots data/cooja_clean_5seed/node_snapshots.csv \
--output results/cooja_clean_5seed_validation_selected \
--models dqn ddqn dueling_ddqn agg_gcn ml_gcn attn_ml_gcn \
--epochs 350 \
--hidden-dim 48 \
--learning-rate 0.02 \
--positive-class-weight 2 \
--threshold-metric f1
```

5.4 Run Layer Analysis and Propagation Sensitivity

```
python scripts/analyze_layers.py \
--snapshots data/cooja_clean_5seed/node_snapshots.csv \
--output results/cooja_clean_5seed_layer_analysis

python scripts/run_graph_sensitivity.py \
--snapshots data/cooja_clean_5seed/node_snapshots.csv \
--output results/cooja_clean_5seed_graph_sensitivity \
--steps 1,2
```

5.5 Generate Tables and Figures

```
python scripts/make_tables.py results/cooja_clean_5seed_full_comparison
python scripts/make_figures.py results/cooja_clean_5seed_full_comparison
```